

LinkCommerce — Documento de Análise de Projeto

Data: Julho 2026
Versão analisada: 0.1.0 (MVP)
Stack: Next.js 14 · Prisma · PostgreSQL (Neon) · Zustand · Tailwind CSS · NextAuth v5

1. ESTADO ATUAL — VISÃO GERAL

Módulo	Estado	Prioridade
Autenticação	✅ Completo	—
Dashboard lojista	✅ Funcional	Melhorias
Storefront pública	✅ Funcional	Melhorias
POS offline	✅ Funcional	—
Internacionalização (4 idiomas)	✅ Completo	—
Multi-moeda (EUR/USD/AOA)	✅ Completo	—
Checkout / Pagamento online	❌ Não implementado	Crítico
Email / Notificações	❌ Não implementado	Crítico
Upload de imagens	❌ Não implementado	Crítico
Subscrição de planos (billing)	❌ Não implementado	Crítico
Variantes de produto (UI)	⚠️ Schema pronto, UI incompleta	Alta
Formulário de contacto (backend)	⚠️ Frontend pronto, sem envio	Alta
Admin dashboard (detalhes)	⚠️ Básico funcional	Média

2. ERROS E PROBLEMAS IDENTIFICADOS

2.1 Erros Críticos de Negócio

❌ Sem Checkout Funcional

O botão "Finalizar compra" no `CartDrawer` não tem handler. O cliente pode adicionar produtos ao carrinho mas **não consegue comprar**. Isto torna a storefront decorativa — sem receita possível.

- Ficheiro: `components/storefront/cart-drawer.tsx` linha 128
- Rota em falta: `/api/checkout` e `/checkout` (página de conclusão)
- Sem integração Stripe, MBWay ou Multicaixa

❌ Formulário de Contacto Sem Backend

O formulário em `/loja/[subdominio]/contacto` renderiza campos mas não envia nada. Submeter o form resulta num reload silencioso.

- Ficheiro: `app/loja/[subdominio]/contacto/page.tsx`

- Sem `action` na `<form>`, sem Server Action, sem email

✗ Subscrição de Planos Quebrada

A página de planos tem um formulário que faz POST para `/api/checkout/subscricao` — essa rota **não existe**. Qualquer clique em "Upgrade" gera um 404.

- Ficheiro: `app/(dashboard)/dashboard/configuracoes/planos/page.tsx` linha 44
- Rota inexistente: `/api/checkout/subscricao`

✗ Upload de Imagens por URL Manual

O campo de imagem dos produtos aceita apenas uma string URL. Imagens de Pinterest, Revelbee ou outras páginas de produto **não são URLs directas de imagem** e não renderizam. O lojista não tem forma fácil de adicionar imagens reais.

- Ficheiro: `components/dashboard/produto-form-dialog.tsx`
- Sem upload S3/Cloudinary/R2
- Sem validação/preview antes de guardar

2.2 Erros de Experiência (UX)

⚠ Variantes Sem UI de Edição

O schema Prisma tem modelo `Variante` (cor, tamanho, preço extra, stock). O dashboard mostra o número de variantes por produto mas não tem interface para criar, editar ou eliminar variantes. Um lojista de calçado (como "glow shoes") não consegue gerir tamanhos 38–45.

⚠ Stock Não Decrementado em Pedidos Online

O endpoint `/api/pos/sync` decrementa stock (POS offline). Mas não existe endpoint de checkout online, logo pedidos online nunca decrementam stock — problema de consistência de inventário.

⚠ Sem Paginação em Produtos, Pedidos e Clientes

As listas de produtos, pedidos e clientes no dashboard fazem `findMany()` sem limite. Com catálogos grandes (500+ produtos) o tempo de carregamento pode tornar o dashboard inutilizável.

⚠ Cores da Loja Não Validadas

O campo `corPrimaria` aceita qualquer string. Se um lojista colocar um valor inválido, todo o CSS inline da storefront quebra silenciosamente.

⚠ Sem Indicador de Estado do Pedido para o Cliente

O cliente faz a compra (quando existir) mas não recebe confirmação visual nem email. O lojista vê os pedidos no dashboard, mas o cliente fica sem feedback.

⚠ Sem Página de Produto Individual

A storefront lista produtos numa grid mas não tem rota `/loja/[subdominio]/produto/[id]`. Não há página de detalhe, galeria de imagens, descrição completa ou selecção de variantes num contexto dedicado.

2.3 Problemas de Segurança

● Credenciais da Base de Dados no Repositório

O ficheiro `.env` contém a `DATABASE_URL` com credenciais reais do Neon (username, password, host). Se este repositório for committed para um repositório público (GitHub), as credenciais ficam expostas.

Ação imediata: Adicionar `.env` ao `.gitignore` (verificar se já está), e rotacionar a password no painel Neon.

● **AUTH_SECRET Exposto no .env**

O segredo JWT do NextAuth está visível. Em produção deve ser definido apenas como variável de ambiente no servidor, nunca no repositório.

● **Sem Rate Limiting nas Rotas de Auth**

`/api/register` e `/entrar` não têm protecção contra brute-force. Um atacante pode tentar milhares de passwords sem bloqueio.

● **Sem CSRF Protection no POS Sync**

`/api/pos/sync` não valida origem. Embora a rota esteja protegida por autenticação, requests cross-site não autenticados poderiam ser tentados.

2.4 Problemas Técnicos / Performance

⚠ **Sem Índice em `Pedido.createdAt`**

As queries de pedidos ordenam por `createdAt DESC` mas não há índice declarado no schema para este campo. Em lojas com muitos pedidos, a ordenação faz full table scan.

⚠ **`getLocale()` Chamado Múltiplas Vezes por Request**

`getLocale()` chama `cookies()` e `headers()` do Next.js. Nas páginas com múltiplos componentes server, é chamado repetidamente. Deveria ser calculado uma vez no layout e passado como prop.

⚠ **Imagens sem `next/image` Optimização no CartDrawer**

O `cart-drawer.tsx` usa `<img>` nativo (com `eslint-disable`) em vez de `next/image`. Sem optimização automática de formato e tamanho.

⚠ **Sem Loading States no Dashboard**

O dashboard faz fetch directamente (Server Components), sem loading skeletons adequados. O utilizador vê uma página em branco durante o carregamento.

3. O QUE FALTA IMPLEMENTAR

3.1 Funcionalidades Bloqueadoras (sem estas, não existe produto)

● **Sistema de Pagamento Online**

O que falta:

- Rota `/api/checkout` — criar sessão de pagamento
- Página `/checkout` — resumo da compra + formulário morada
- Página `/checkout/sucesso` — confirmação + número de pedido
- Integração Stripe (pagamento com cartão internacional)
- Integração MBWay (Portugal/Angola — telemóvel)

- Integração Multicaixa Express (Angola)
- Webhook Stripe para confirmar pagamento e criar `Pedido` na BD
- Decremento de stock após pagamento confirmado

● Sistema de Email Transaccional

O que falta:

- Biblioteca: Resend ou Nodemailer com SMTP
- Template: confirmação de registo
- Template: confirmação de pedido (cliente)
- Template: novo pedido (lojista)
- Template: mudança de status do pedido
- Template: reset de password
- Template: resposta ao formulário de contacto

● Upload de Imagens

O que falta:

- Integração Cloudflare R2 ou AWS S3 ou Cloudinary
- Rota `/api/upload` (multipart/form-data)
- Interface drag-and-drop no `produto-form-dialog.tsx`
- Preview da imagem antes de guardar
- Compressão automática para WebP

● Sistema de Billing / Subscrições

O que falta:

- Rota `/api/checkout/subscricao` — criar sessão Stripe para plano mensal/anual
- Webhook Stripe para activar/desactivar subscrição (`StatusSubscricao`)
- Lógica de `limiteProdutos` por plano (bloquear criação de produto além do limite)
- Portal do cliente Stripe (para o lojista gerir cartão de crédito)
- Envio de factura por email

3.2 Funcionalidades de Alta Prioridade

● Página de Produto Individual

Rota: `/loja/[subdominio]/produto/[id]`

- Galeria de imagens
- Descrição completa
- Selecção de variante (tamanho, cor)
- Botão adicionar ao carrinho
- Produtos relacionados
- Reviews do produto

● UI de Variantes no Dashboard

- Secção expandível no `produto-form-dialog.tsx`
- Adicionar/remover variantes (nome, preço extra, stock)
- Tabela de stock por variante

● Gestão de Pedidos — Lojista

- Actualizar status do pedido (PENDING → PROCESSING → SHIPPED)

- Inserir número de tracking
- Imprimir etiqueta (básico)
- Notificação automática ao cliente por email quando status muda

● **Gestão de Envios**

- Configurar métodos de envio (taxa fixa, por peso, recolha)
- Integrar preços de envio no checkout

● **Domínio Próprio**

O schema tem `Loja.dominioProprio` mas não há mecanismo para:

- DNS verification
 - Certificado SSL automático (Let's Encrypt)
 - Middleware de resolução de domínio próprio
-

3.3 Funcionalidades de Média Prioridade

● **Analytics / Relatórios**

- Vendas por período (gráfico)
- Produtos mais vendidos
- Clientes recorrentes vs. novos
- Taxa de conversão storefront
- Origem do tráfego

● **Campanha de Marketing**

- Email de carrinho abandonado (recuperação)
- Cupões com limite por utilizador
- Newsletter opt-in no checkout
- Pop-up de desconto (first purchase)

● **Motor de Pesquisa na Storefront**

- Campo de pesquisa por nome de produto
- Filtros por categoria / preço
- Ordenação (mais barato, mais recente, mais vendido)

● **Registo de Cliente no Checkout**

- Conta cliente com histórico de pedidos
- Endereços guardados
- Acompanhamento de encomendas

● **API Pública (para plano Enterprise)**

- REST API documentada para integração externa
 - Webhook de pedidos para sistemas externos (ERP, CRM)
 - API key management no dashboard
-

3.4 Funcionalidades Adicionais (Nice-to-Have)

- **Multi-imagem por produto** (galeria)
- **Importação CSV** de catálogo de produtos
- **QR Code** da loja para partilha
- **SEO automático** (meta tags dinâmicas por produto)
- **Sitemap.xml** gerado automaticamente

- **Open Graph** images por produto
 - **Avaliações reais** de clientes com verificação de compra
 - **Chat ao vivo** integrado (Tawk.to ou Intercom embed)
 - **Notificações push** para lojista (novo pedido)
 - **App móvel** para POS (React Native + mesma API)
-

4. SUGESTÕES DE MELHORIA

4.1 Arquitectura e Código

Centralizar `getLocale()` no Layout

Em vez de chamar `getLocale()` em cada page, calcular no layout raiz e passar via `searchParams` ou como prop explícita. Reduz overhead de cookies/headers por request.

Paginação Universal com Cursor

Implementar cursor-based pagination em todas as listagens (produtos, pedidos, clientes). Mais eficiente que offset para tabelas grandes e funciona bem com Prisma.

Validação de Cor Primária

Adicionar validação de hex color no form de configurações. Um regex simples (`/^[0-9A-Fa-f]{6}$/`) evita CSS quebrado.

Error Boundaries Mais Granulares

O projecto tem `error.tsx` no root mas não por página crítica. Erros no dashboard de produtos não devem quebrar toda a sidebar.

Abstracção de Server Actions

As Server Actions em `dashboard/produtos/actions.ts` repetem lógica de autorização (`getLojaId`). Criar um wrapper `withAuth(action)` reduziria boilerplate.

4.2 Experiência do Utilizador

Onboarding Guiado

Após criar a loja, o lojista deveria ter um checklist guiado:

1. ☒ Loja criada
2. ☐ Adicionar primeiro produto
3. ☐ Configurar método de pagamento
4. ☐ Publicar loja
5. ☐ Partilhar link

Preview em Tempo Real nas Configurações

Ao alterar `corPrimaria` nas configurações, mostrar uma preview miniatura da storefront sem recarregar a página.

Indicador de Plano no Dashboard

Mostrar no topo do dashboard: "Plano Free — 2/3 produtos utilizados. Fazer upgrade >"

Tour Interactivo (Primeira Visita)

Na primeira sessão após onboarding, mostrar tooltips sequenciais a apontar para cada secção do dashboard.

4.3 Design e Branding

Consistência de Cores no Dashboard

O dashboard usa `bg-slate-50` e cards brancos genéricos. Poderia incorporar subtilmente a cor primária da plataforma (#153DFC) nos elementos de acção principal (botões, badges activos).

Dark Mode

O dashboard está totalmente em light mode. Um dark mode opcional melhoraria a experiência para lojistas que trabalham à noite (contexto POS).

Tipografia

O projecto referencia `font-montserrat` na página de login mas a fonte não é importada de forma consistente. Definir a fonte principal no `layout.tsx` raiz com `next/font`.

Skeleton Loading

Substituir páginas em branco durante fetch por skeletons animados — melhora percepção de velocidade.

4.4 Segurança e Compliance

Política de Privacidade e Termos de Serviço

Necessários legalmente (RGPD em Portugal, LGPD em Angola/Brasil) antes de aceitar utilizadores reais.

Cookie Consent

O cookie `LC` (idioma) e qualquer cookie de analytics necessita de banner de consentimento RGPD.

Dados Pessoais

O modelo `Cliente` guarda email, nome, telefone e morada. Necessário:

- Direito ao esquecimento (endpoint delete)
- Exportação de dados pessoais
- DPO (Data Protection Officer) mencionado na política

Logs de Auditoria

Acções de admin (impersonation, suspend loja) devem ser registadas com timestamp, user e IP.

5. ROTEIRO PARA LANÇAMENTO NO MERCADO

Fase 1 — Fechar o Produto (4–6 semanas)

Objectivo: ter uma loja completamente funcional do ponto de vista do cliente

#	Tarefa	Estimativa
1	Integrar Stripe (checkout + webhook)	1 semana
2	Página de checkout + confirmação	3 dias
3	Email transaccional com Resend	3 dias
4	Upload de imagens com Cloudflare R2	2 dias
5	UI de variantes no dashboard	2 dias
6	Página de produto individual	3 dias
7	Gestão de status de pedido pelo lojista	2 dias

8	Correção do billing de planos	2 dias
---	-------------------------------	--------

Fase 2 — Estabilizar e Escalar (3–4 semanas)

Objectivo: produto pronto para primeiros 100 lojistas reais

#	Tarefa	Estimativa
1	Rate limiting nas rotas de auth	1 dia
2	Paginação em listagens (produtos, pedidos, clientes)	2 dias
3	Testes end-to-end (Playwright) — fluxo compra	3 dias
4	Monitorização de erros (Sentry)	1 dia
5	Analytics básicos no dashboard (Chart.js ou Recharts)	3 dias
6	SEO: meta tags dinâmicas + sitemap.xml	2 dias
7	Onboarding checklist para novos lojistas	2 dias
8	Domínio próprio (CNAME + verificação)	1 semana

Fase 3 — Crescimento (ongoing)

Objectivo: produto diferenciado e defensável no mercado

#	Tarefa	Prioridade
1	Integrações de envio (CTT, DHL, Correios dos Correios)	Alta
2	Relatórios e analytics avançados	Alta
3	Recuperação de carrinhos abandonados (email)	Alta
4	API pública documentada (Swagger)	Média
5	App móvel POS (React Native)	Média
6	White-label (plano Enterprise)	Média
7	Marketplace de templates	Baixa
8	Programa de afiliados para lojistas	Baixa

6. ANÁLISE DE MERCADO E POSICIONAMENTO

Público-Alvo Primário

- Pequenos negócios em Portugal** — cafés, boutiques, artesãos, prestadores de serviços que querem vender online sem fricção técnica
- Negócios angolanos** — mercado sub-servido com poucos players locais; suporte a Multicaixa e AOA é diferenciador real

3. **Negócios da diáspora lusófona** — empreendedores em Portugal/França/Brasil com fornecedores em Angola/Moçambique

Vantagens Competitivas Actuais

Vantagem	Detalhes
POS offline-first	Poucos SaaS e-commerce têm POS integrado e offline-first com sync automático
Multi-moeda nativa	EUR + USD + AOA out-of-the-box; mercado angolano negligenciado pela concorrência
4 idiomas incluídos	PT/EN/FR/ES desde o dia 1 — mercado lusófono + francófono (África Ocidental)
Stack moderno	Next.js 14 App Router — performance e SEO superiores a concorrentes legacy
IA integrada	Geração de descrições de produto com Claude — feature visível e valorizada por lojistas não-técnicos

Concorrência e Diferenciação

Concorrente	Força	Fraqueza LinkCommerce deve explorar
Shopify	Ecossistema, extensões	Caro, inglês-first, sem Multicaixa
WooCommerce	Flexibilidade	Complexo, requer WordPress, sem POS integrado
Loja Integrada	Mercado BR	Sem suporte Angola, sem POS offline
Jumia	Marketplace Angola	Não é uma plataforma própria — sem branding do lojista
Vendus (PT)	POS Portugal	Sem storefront pública, sem multi-moeda

Modelo de Receita Recomendado

Free	→ 0€/mês	3 produtos, comissão 3%	→ Aquisição
Starter	→ 19€/mês	50 produtos, comissão 1%	→ Small business
Growth	→ 49€/mês	500 produtos, sem comissão	→ Faturação séria
Enterprise	→ 199€/mês	Ilimitado + domínio + API	→ Marcas

Receita adicional:

- Comissões transaccionais (plano Free/Starter)
- Setup fee opcional (migração de loja Shopify/WooCommerce)
- Serviços geridos (configuração + design da loja)

7. ACÇÕES IMEDIATAS RECOMENDADAS

Ordenadas por impacto e urgência:

Esta Semana

- 1. Rotacionar credenciais — nova password Neon + novo AUTH_SECRET
- 2. Verificar .gitignore — garantir que .env não é committed
- 3. Desligar a rota de planos temporariamente (evitar 404 que frustra utilizadores)

Próximas 2 Semanas

- 4. Integrar Stripe — é o único bloqueador real para existir receita
- 5. Integrar Resend — email de confirmação de pedido é o mínimo de confiança para o cliente
- 6. Upload de imagens — sem imagens reais, a storefront não convence

Próximo Mês

- 7. Beta fechado — convidar 10–20 lojistas reais para testar e dar feedback
- 8. Corrigir bugs reportados pelo beta antes de abrir ao público
- 9. Configurar Sentry — monitorizar erros em produção
- 10. Landing page de waitlist — recolher emails antes do lançamento público

8. ESTIMATIVA TÉCNICA DE ESFORÇO

Para um programador sénior full-stack:

Módulo	Complexidade	Estimativa
Stripe Checkout completo	Alta	1 semana
Email com Resend	Média	2–3 dias
Upload Cloudflare R2	Média	2 dias
UI variantes produto	Média	2 dias
Página produto individual	Baixa/Média	3 dias
Gestão pedidos (status + email)	Média	3 dias
Billing de planos (Stripe Sub)	Alta	1 semana
Paginação universal	Baixa	1 dia
Rate limiting	Baixa	1 dia
Sentry + monitorização	Baixa	4 horas
SEO + sitemap	Baixa	1 dia
Testes end-to-end	Alta	1 semana
Total estimado		~6–8 semanas

Documento gerado em Julho 2026 com base em análise estática do código-fonte e base de dados do projecto LinkCommerce.